# Density Scene-Allocation Specification

Deterministic conversion of one browser face budget into canonical shell targets

mdstats

2026-07-22

## Contents



## 1 Purpose and status

**Module:** `mdstats.plotting.density_scene_budget`
**Status:** normative and implemented.

This module allocates an initial canonical face target to every requested density shell. It is a pure planning module: no mesh is extracted, simplified, recontoured, or serialized here.

## 2 Normative ownership

The module owns:

- `DensitySceneShellRequest`;
- `DensitySceneAllocationOptions`;
- `DensitySceneShellAllocation`;
- `DensitySceneBudgetPlan`;
- deterministic largest-remainder apportionment under display replication.

It does not own final compliance. Actual output may miss its initial target and is handled by `density_scene_fit`.

## 3 Inputs

```
DensitySceneShellRequest(
    shell_key: str,
    field_key: str,
    label: str,
```

```
    mass_fraction: float,
    selected_node_count: int,
    display_replication: int = 1,
    visual_importance: float = 1.0,
    max_canonical_faces: int = 250_000,
)
```

`selected_node_count` is a backend-neutral proxy for occupied volume. The default allocation weight is

$$w_s = N_s^{2/3} I_s,$$

where $N_s$ is selected-node count and $I_s$ is visual importance.

```
DensitySceneAllocationOptions(
    min_canonical_faces_per_shell: int = 4_000,
    shell_importance: tuple[float, ...] = (1.0, 0.72, 0.48),
    reserve_face_fraction: float = 0.15,
)
```

# 4 Algorithm

1. Validate unique shell keys and positive replication multiplicities.
2. Reserve the minimum canonical allocation for every shell.
3. Convert minimum canonical counts to serialized counts.
4. Reserve the declared scene fraction for fitting overshoot and non-density work.
5. Weight the remainder by $w_s$.
6. Apply per-shell canonical maxima.
7. Use deterministic largest-remainder apportionment.
8. Return a plan whose serialized allocation never exceeds the budget.

For allocation $A_s$ and replication $m_s$:

$$A_{s,\text{serialized}} = m_s A_s.$$

# 5 Output

```
DensitySceneBudgetPlan(
    budget,
    requests,
    allocations,
    allocated_serialized_faces,
    unallocated_serialized_faces,
)
```

Request and allocation order are identical and deterministic.

# 6 Failure semantics

Raise `GraphComplexityError` when minimum reserves alone exceed the browser face budget. Do not silently drop shells or lower their minimum below the declared policy.

A later target miss is not a planning failure.

# 7 Edge cases

- One shell is valid.

- Repeated shell keys are invalid.
- Empty shell requests are invalid for scene rendering.
- A shell maximum below the policy minimum is capped consistently or rejected before apportionment.
- Display replication must be counted during allocation, not after it.

# 8 Focused validation

Tests must cover:

- request-order-independent allocation;
- exact serialized totals;
- reserve handling;
- maximum caps;
- insufficient-minimum failure;
- JSON round trips;
- three HDR importance levels;
- mixed replication multiplicities.